

1 · Motivation

CORE INSIGHT

GUI-only agents perform every action through clicks & keystrokes — even when a single line of code would suffice. **Brittle** and **inefficient** on long-horizon tasks.

GUI-only problems:

✗ Long & intricate click sequences for file / data tasks

✗ Visual grounding ambiguity (similar icons & menus)

✗ Cumulative error probability over long horizons

✗ A single mis-click can derail the entire task

Our proposal:

✓ Add **coding** as a first-class action

✓ Hybrid policy: programmatic + visual control

✓ Orchestrator dynamically picks modality per subtask

CONTRIBUTION

First multi-agent system unifying **GUI** + **Code**

2 · Problem Formulation

Interactive decision-making with policy $\pi(a_t \mid H_t, G)$ over a **hybrid action space**:

$$\mathcal{A} = \mathcal{A}_{\text{GUI}} \cup \mathcal{A}_{\text{Code}}$$

► $\mathcal{A}_{\text{GUI}}$: clicks, typing, scrolling, dragging

► $\mathcal{A}_{\text{Code}}$: Python / Bash scripts on the OS

Hierarchical: meta-policy π_{Orch} delegates each subtask.

3 · CoAct-1 Architecture

Figure 1. Multi-agent system design. An **Orchestrator** decomposes the user task and delegates each subtask to either a **Programmer** (multi-round coding) or a **GUI Operator** (vision-based UI actions).

Three **specialized agents** with **isolated memories** collaboratively generate trajectory τ to solve goal G . Executor memory is cleared after every subtask.

4 · Workflow & Memory

1 **Plan**. Orchestrator observes screenshot + history, decomposes G .

2 **Delegate**. Meta-policy picks Programmer or GUI Operator per subtask.

3 **Execute**. Selected agent runs its own multi-round loop with the OS.

4 **Summarize**. Dedicated summarizer condenses the execution trace.

5 **Handoff**. Summary + screenshot return to Orchestrator; executor memory cleared.

6 **Terminate**. Orchestrator verifies goal completion & cross-checks results.

HIERARCHICAL MEMORY Orchestrator holds long-term H_t ; executors keep short-term working memory *reset after each subtask* — preventing cross-contamination.

5 · Main Results — New State of the Art

60.8%
OSWORLD SR

52.5%
WINDOWSAGENTARENA

10.15
AVG STEPS / TASK

Table 1 · OSWorld (369 tasks) — Success Rate %

METHOD	OFFICE	DAILY	PROF.	OS	MULTI	AVG.
OpenAI CUA 4o	25.0	39.2	55.4	58.3	18.5	31.4
OpenCUA (32B)	30.1	38.9	60.7	52.2	18.1	33.8
Jedi-7B w/ o3	47.9	64.4	75.9	50.0	35.3	51.0
GTA-1-7B w/ o3	55.7	64.7	61.2	62.5	38.3	53.1
Agent S2.5 w/ o3	54.2	55.8	75.4	75.0	44.1	56.0
★ CoAct-1 (ours)	64.8	66.5	71.8	75.0	47.9	60.8

100–150 step budget. CoAct-1 wins 5 / 6 categories.

Table 2 · WindowsAgentArena (154 tasks) — SR %

METHOD	OFFICE	WEB	SYS.	VSCODE	VLC	AVG.
Agent S	0.0	13.3	45.8	29.2	19.1	18.2
NAVI	0.0	27.3	33.3	27.3	30.3	19.5
Agent S2	7.0	16.4	54.2	62.5	28.6	29.8
★ CoAct-1 (ours)	30.4	50.0	83.3	62.5	47.2	52.5

Transfers to Windows: +22.7 pts over Agent S2.

Figure 3a · Avg. Steps per Solved Task (100-step) — 33% fewer steps than GTA-1

Method	Avg. Steps
GTA-1	15.2
UI-TARS	14.9
CUA 4o	6.1
CoAct-1	10.2

6 · Ablation: Backbone Models

Table 3 · Role of each backbone (OSWorld)

ORCHESTRATOR	PROGRAMMER	GUI OP.	SR %
o4-mini	o4-mini	CUA 4o	43.4
o3	o3	CUA 4o	58.7
o3	o4-mini	CUA 4o	60.8

KEY FINDING Upgrading Orchestrator to o3 yields +15.3 pts. Best: *heterogeneous* backbones — o3 for planning, o4-mini for coding.

7 · Ablation: Hybrid vs Single Modality

Table 4 · Single-modality vs Hybrid (OSWorld %)

CONFIG	OFFICE	OS	MULTI	AVG.	STEPS
Programmer only	40.9	62.5	29.6	35.7	1.14
GUI Op. only	43.5	79.2	35.7	50.7	11.2
★ CoAct-1 (both)	64.8	75.0	47.9	60.8	10.2

► **Programmer-only: 1.14 steps** (35.7%) — limited scope.

► **GUI-only: 11.2 steps** (50.7%) — brittle chains.

► **Full hybrid: +25.1 pts** over coding, **+10.1 pts** over GUI.

SYNERGY, NOT SUBSTITUTION Dynamic per-subtask modality selection is key.

8 · Takeaways

✓ New SOTA on OSWorld (60.8%) & WAA (52.5%)

✓ Coding as first-class action complements GUI

✓ 33% fewer steps than leading GUI-only agents

✓ Generalizes across Ubuntu & Windows